

Snippets `app/main.py` — URI Pack v1 (PMV) + Prompts para agente IA

Objetivo: cablear los routers y modelos del **URI Pack v1 (PMV)** sin romper nada existente.
Backend: FastAPI + SQLAlchemy.

0) Prerrequisitos y orden

1. **Roadmap URIs (Core)** implementado (tabla `entity_uri_registry`, hooks, resolver, subrecursos vacíos).
2. **URI Pack v1 (PMV)** archivos creados (shortlinks, embeds `listen`, analytics, claims, public_ld, utils, schemas) + **migraciones preparadas**.
3. **Este cableado** en `app/main.py` (imports + include_router + registro de modelos).
4. **Aplicar migraciones y instalar deps** (`qrcode[pil]`).
5. **Ejecutar smoke tests**.

1) Snippets listos para pegar en `app/main.py`

1.1 Imports de routers (poner junto al resto de imports de routers)

```
from app.routers import shortlinks as shortlinks_router
from app.routers import embeds as embeds_router
from app.routers import analytics as analytics_router
from app.routers import claims as claims_router
from app.routers import public_ld as public_ld_router
```

1.2 Forzar registro de modelos (antes de `Base.metadata.create_all(...)` o antes del arranque del servidor)

```
# Registrar modelos nuevos para que se creen las tablas (no remover)
from app import models_shortlinks, models_analytics, models_claims # noqa:
F401
```

1.3 Incluir routers (cerca de donde ya haces otros `include_router`)

```
app.include_router(shortlinks_router.router, tags=["Shortlinks"])
app.include_router(embeds_router.router, tags=["Embeds"])
app.include_router(analytics_router.router, tags=["Analytics"])
app.include_router(claims_router.router, tags=["Claims"])
app.include_router(public_ld_router.router, tags=["Public JSON-LD"])
```

Notas:

- No agregues `prefix` aquí; cada router ya define el suyo (`/embed`, `/public`, etc.).
- Si usas Alembic, estos imports igual deben quedar para evitar modelos "zombis" en ejecución.

2) Prompt breve para agente IA (aider / Continue / Code Assist)

Copiar/pegar en tu agente. Ajusta si tu archivo principal no es `app/main.py`.

Tarea: Cablear el URI Pack v1 (PMV) en `app/main.py` sin romper nada existente.

1) Agrega estos imports de routers cerca de los otros imports de routers:

```
from app.routers import shortlinks as shortlinks_router
from app.routers import embeds as embeds_router
from app.routers import analytics as analytics_router
from app.routers import claims as claims_router
from app.routers import public_ld as public_ld_router
```

2) Antes de cualquier ``Base.metadata.create_all(...)`` o del arranque del servidor, agrega:

```
from app import models_shortlinks, models_analytics, models_claims # noqa:
F401
```

3) Agrega estas líneas donde ya haces otros ``app.include_router(...)``:

```
app.include_router(shortlinks_router.router, tags=["Shortlinks"])
app.include_router(embeds_router.router, tags=["Embeds"])
app.include_router(analytics_router.router, tags=["Analytics"])
app.include_router(claims_router.router, tags=["Claims"])
app.include_router(public_ld_router.router, tags=["Public JSON-LD"])
```

Condiciones:

- No tocar otras rutas ni middlewares existentes.
- Mantener estilo de importación actual.
- Validar que el archivo compile.

Checklist de verificación rápida:

- El proyecto levanta sin errores de importación.
- ``GET /embed/{gid}/listen`` responde 200 con HTML.
- ``POST /tools/shortlinks`` funciona y ``GET /s/{code}`` devuelve 302.
- ``GET /public/{tipo}/{gid}/ld.json`` responde 200 con JSON.

3) Diff estilo patch (opcional, si querés aplicar con `git apply`)

Ejemplo orientativo^{**}: ajusta el contexto según tu `main.py` real.

```

--- a/app/main.py
+++ b/app/main.py
@@
+from app.routers import shortlinks as shortlinks_router
+from app.routers import embeds as embeds_router
+from app.routers import analytics as analytics_router
+from app.routers import claims as claims_router
+from app.routers import public_ld as public_ld_router
+
+# Registrar modelos nuevos para creación de tablas
+from app import models_shortlinks, models_analytics, models_claims # noqa:
F401
@@
    app.include_router(existing_router.router)
+app.include_router(shortlinks_router.router, tags=["Shortlinks"])
+app.include_router(embeds_router.router, tags=["Embeds"])
+app.include_router(analytics_router.router, tags=["Analytics"])
+app.include_router(claims_router.router, tags=["Claims"])
+app.include_router(public_ld_router.router, tags=["Public JSON-LD"])

```

4) Smoke tests (rápidos y completos)

```

# 0) Levantar el servidor
uvicorn app.main:app --reload

# 1) Shortlinks – crear
curl -s -X POST http://127.0.0.1:8000/tools/shortlinks \
  -H 'Content-Type: application/json' \
  -d '{"gid":"TEST_GID","target_uri":"https://soup.market/band/banda-
nam-x7k3"}'
# Copiá el "code" devuelto (ej. ABCD1234) y probá la redirección (302):
curl -i http://127.0.0.1:8000/s/ABCD1234 | head -n 20
# Ver QR PNG en el navegador:
open http://127.0.0.1:8000/tools/shortlinks/ABCD1234/qr.png # (macOS)
# o xdg-open en Linux

# 2) Embed listen (HTML 200)
open "http://127.0.0.1:8000/embed/TEST_GID/listen?spotify=https://
open.spotify.com/album/xxx&bandcamp=https://bandcamp.com/album/yyy"

# 3) JSON-LD (band/album)
curl -i http://127.0.0.1:8000/public/band/01J8ABC.../ld.json
curl -i http://127.0.0.1:8000/public/album/01J8DEF.../ld.json

# 4) Analytics (view/click) + summary
curl -s -X POST http://127.0.0.1:8000/public/track -H 'Content-Type:
application/json' \

```

```

-d
'{"entity_gid":"TEST_GID","type":"view","subresource":"listen","referrer":"local"}'
curl -s "http://127.0.0.1:8000/admin/analytics/summary?
gid=TEST_GID&date_from=2025-08-01&date_to=2025-08-31" | jq .

# 5) Claim (request + verify)
# Solicitar verificación (guarda el token devuelto):
curl -s -X POST http://127.0.0.1:8000/claim/request -H 'Content-Type:
application/json' \
-d '{"entity_gid":"TEST_GID","requested_by":"artist@example.com"}' | jq .
# Verificar con el token (reemplazá TOKEN_AQUI):
curl -s -X POST http://127.0.0.1:8000/claim/verify -H 'Content-Type:
application/json' \
-d '{"entity_gid":"TEST_GID","token":"TOKEN_AQUI"}' | jq .

```

Si tu archivo principal no se llama `app/main.py`, indicá la ruta y genero la variante exacta.

5) Migraciones (SQL / Alembic)

Opción A — SQL directo (PostgreSQL)

1. Crear archivo: `migrations/2025-08-12_uri_pack_v1.sql` (usa el contenido ya provisto en el doc del **URI Pack v1**).
2. Ejecutar contra tu DB:

```
psql "$DATABASE_URL" -f migrations/2025-08-12_uri_pack_v1.sql
```

Ajustá `$DATABASE_URL` o usa `psql -h HOST -U USER -d DB -f ...`.

Opción B — Alembic (si usás Alembic)

```

alembic revision -m "uri-pack-v1"
# Pegar el SQL de creación de tablas en upgrade(); opcionalmente define
downgrade().
alembic upgrade head

```

Dependencias

```
pip install qrcode[pil]
```

6) Prompt para agente IA — migrar + smoke tests

Tarea: Completar el cableado de main.py, aplicar migraciones del URI Pack v1 (PMV) y correr smoke tests.

- 1) Verifica que existen los archivos del Pack v1 y que app/main.py contiene:
 - imports de routers (shortlinks, embeds, analytics, claims, public_ld)
 - import de modelos (models_shortlinks, models_analytics, models_claims)
 - include_router(...) de los 5 routers
- 2) Crea migrations/2025-08-12_uri_pack_v1.sql con el SQL de shortlinks, analytics_events y entity_claims.
 - Si hay Alembic, genera una revision y coloca el SQL en upgrade().
 - Ejecuta alembic upgrade head o psql -f según corresponda.
- 3) Instala dependencia: `pip install qrcode[pil]`.
- 4) Arranca el server (uvicorn app.main:app --reload) y corre los smoke tests:
 - Crear shortlink → verificar 302 en /s/{code}
 - /embed/{gid}/listen → 200 con HTML
 - /public/{tipo}/{gid}/ld.json → 200 con JSON
 - /public/track → ok; /admin/analytics/summary → agregados
 - /claim/request y /claim/verify → status verified
- 5) Deja commits atómicos:
 - chore(main): wire-up uri-pack routers & models
 - feat(migrations): uri-pack v1 tables
 - chore(deps): add qrcode[pil]

Condiciones:

- No romper endpoints existentes.
- Verificar que el servicio levante sin errores.
- Reportar códigos HTTP de cada smoke test.